Unsupervised Learning - Clustering

Lecture 5

Unsupervised Learning - Clustering

Learning Objectives:

- Understand the fundamentals of unsupervised learning and clustering.
- Learn different clustering algorithms and how they work.
- Explore the applications of clustering in various fields.
- Understand evaluation metrics for clustering algorithms.
- Implement clustering algorithms in Python with Scikit-learn.

Overview of Unsupervised Learning

In unsupervised learning, the model is trained on a dataset without labeled outputs, allowing it to discover hidden structures or patterns within the data. The goal is to group or cluster the data based on similarities, without predefined categories.

Differences from Supervised Learning

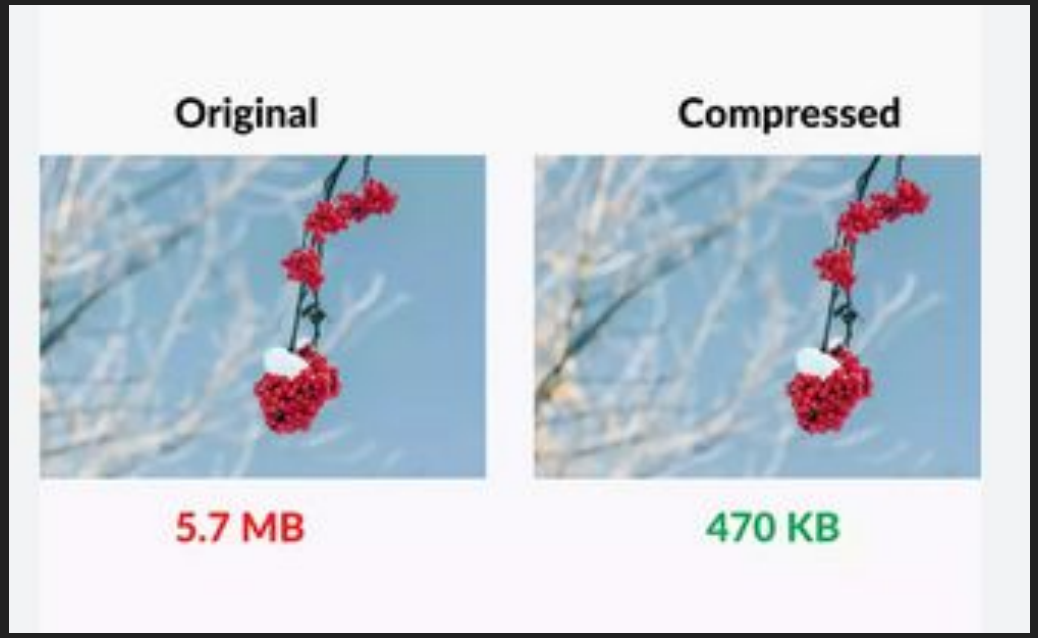
Supervised learning uses labeled data, aiming to map inputs to known outputs, while unsupervised learning works with unlabeled data, focusing on finding patterns or groups.

Applications of Unsupervised Learning

Customer Segmentation: Grouping customers with similar buying behaviors.

Applications of Unsupervised Learning

Image Compression: Reducing image data by clustering similar pixels.

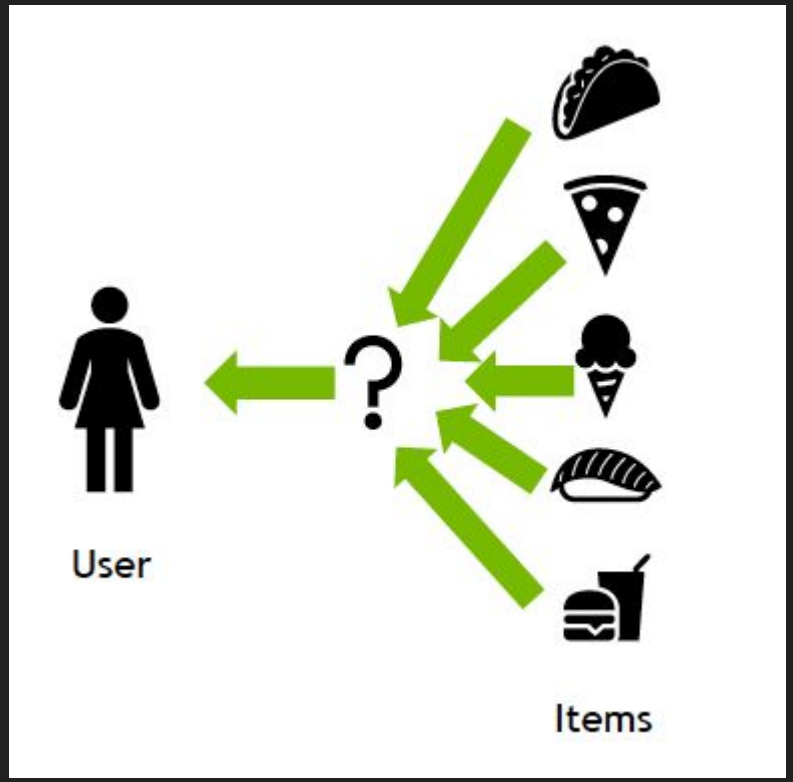


Applications of Unsupervised Learning

Anomaly Detection: Identifying unusual patterns or outliers.

Applications of Unsupervised Learning

Recommender Systems: Finding users with similar preferences to provide recommendations.



Main Techniques in Unsupervised Learning

Clustering

Grouping similar data points together

Main Techniques in Unsupervised Learning

Clustering

Grouping similar data points together

Dimensionality Reduction

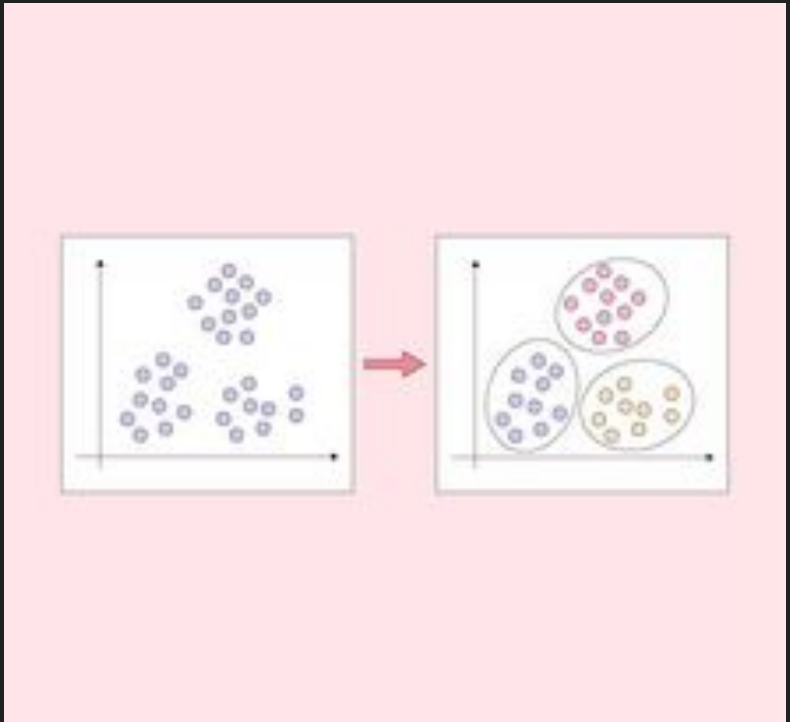
Reducing the number of features in the data while retaining essential information (e.g., PCA - *Principal Component Analysis* (*PCA*) is an unsupervised *learning* algorithm technique used to examine the interrelations among a set of variables.).

What is Clustering?

Clustering is a process of dividing a dataset into distinct groups or clusters so that data points within the same cluster are more similar to each other than to those in other clusters.

Types of Clustering Algorithms

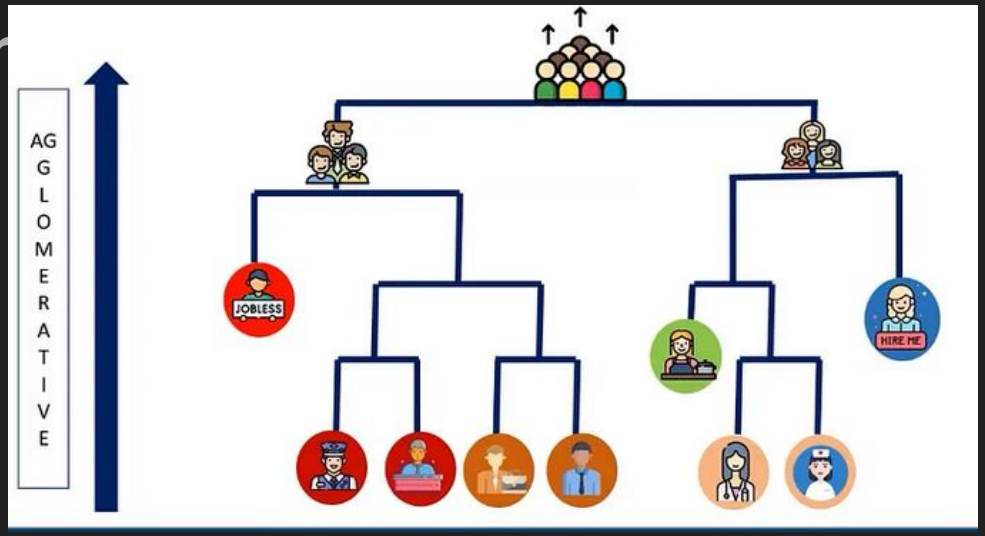
Partitioning Methods: Split data into non-overlapping subsets (e.g., k-means).



Types of Clustering Algorithms

Hierarchical Clusterin

Creates a hierarchy of clusters (e.g., agglomerative clustering).



Types of Clustering Algorithms

Hierarchical clustering is a method of cluster analysis used in data mining. It seeks to build a hierarchy of clusters in a step-by-step manner. There are two main types of hierarchical clustering:

Types of Clustering Algorithms

1. Agglomerative (Bottom-Up Approach):

- **Initial Step:** Starts by treating each data point as a separate cluster. So, if there are N data points, you begin with N clusters.
- **Clustering Process:** In each step, the algorithm merges the two clusters that are closest to each other until all the clusters are merged into one big cluster containing all data points.
- **Dendrogram:** The result can be represented in a tree-like structure called a dendrogram, which shows the arrangement of the clusters and their proximity.

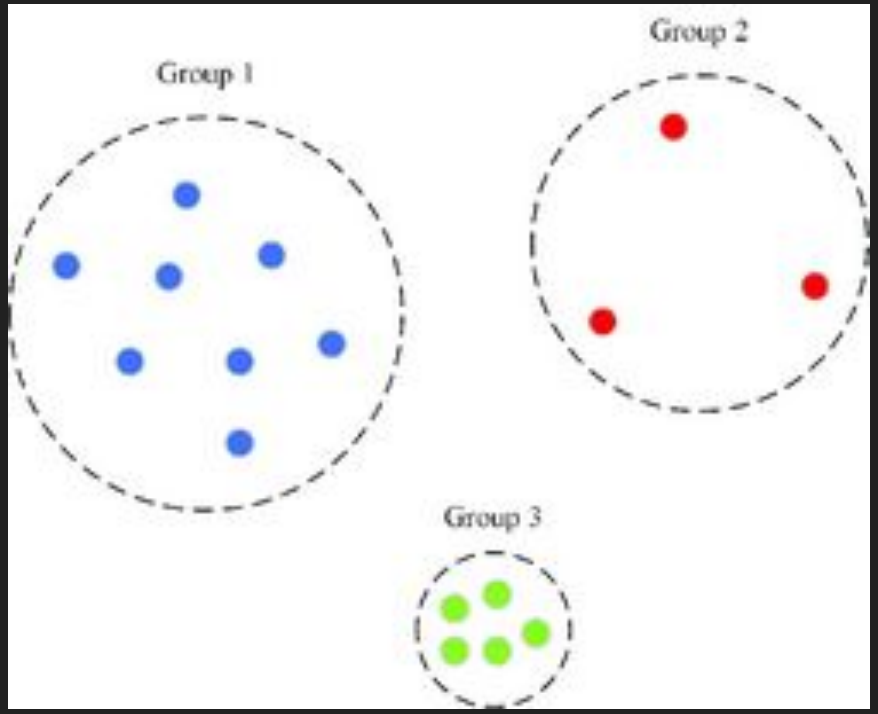
Types of Clustering Algorithms

2. Divisive (Top-Down Approach):

- **Initial Step:** Begins with all data points in a single cluster.
- **Clustering Process:** At each step, the algorithm splits the cluster until each cluster contains only one data point.
- **Top-Down Splitting:** This is less common compared to agglomerative clustering and is computationally more intensive.

Types of Clustering Algorithms

Density-Based Clustering: Forms clusters based on areas of high density (e.g., DBSCAN).

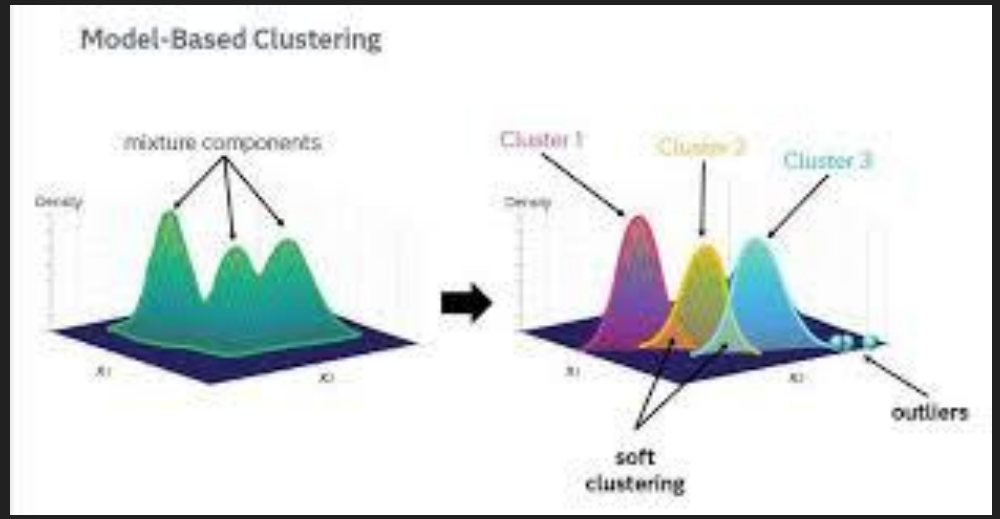


Types of Clustering Algorithms

The Density-based Clustering tool works **by detecting areas where points are concentrated and where they are separated by areas that are empty or sparse**. Points that are not part of a cluster are labeled as noise.

Types of Clustering Algorithms

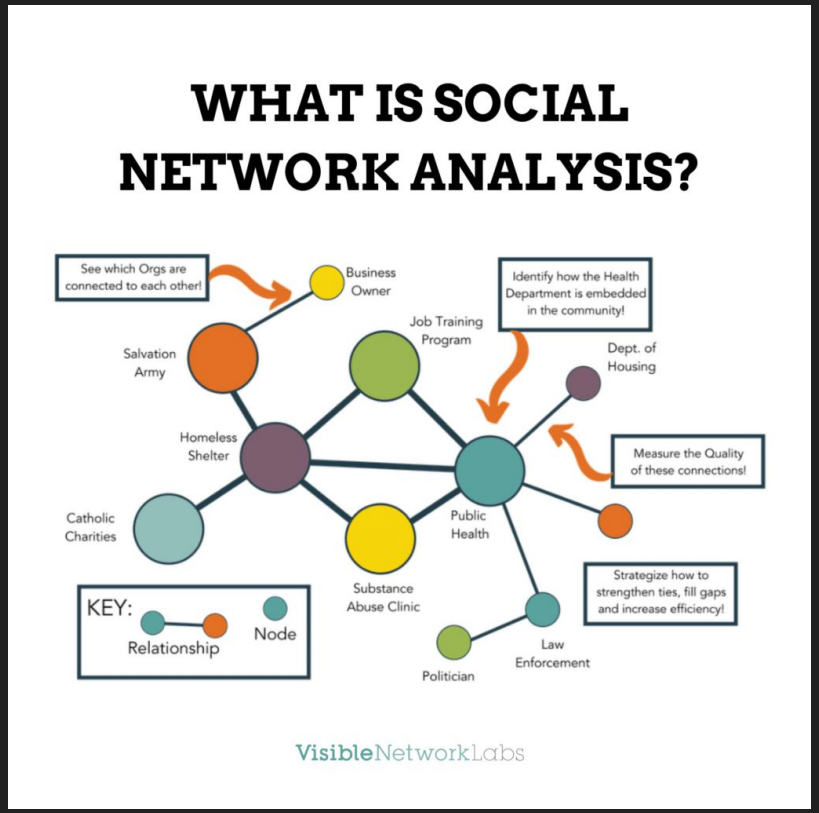
Model-Based Clustering: Uses statistical models to find clusters (e.g., Gaussian Mixture Models).



Applications of Clustering

Market Segmentation: Grouping customers based on purchasing behavior.

Applications of Clustering



Social Network Analysis: Detecting communities or groups.

Applications of Clustering

(**a**) Image

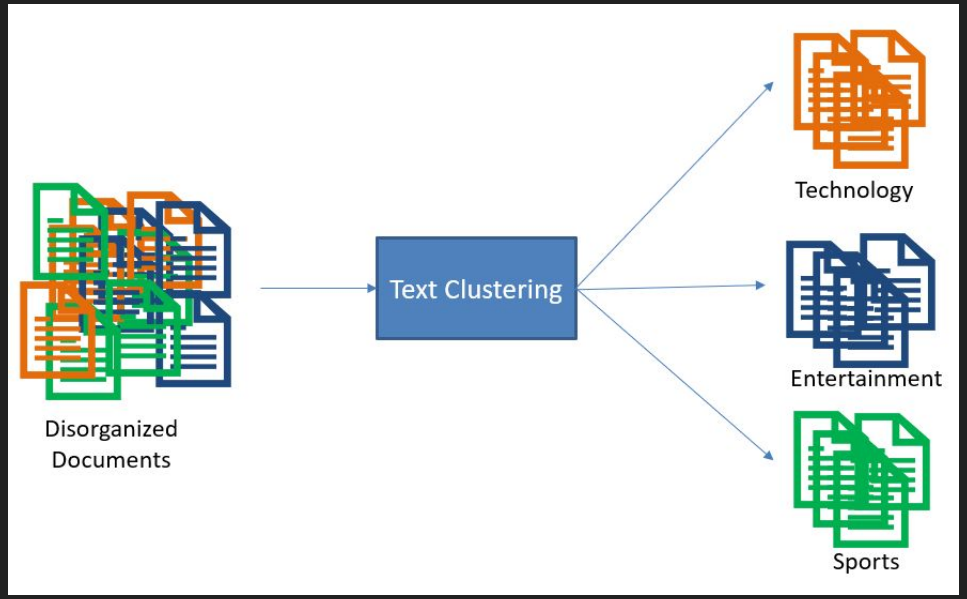
(**b**) Semantic segmentation

(**c**) Instance segmentation

(**d**) Panoptic segmentation

Image Segmentation: Dividing an image into meaningful parts.

Applications of Clustering



Document Clustering: Grouping similar documents for categorization or topic modeling.

Common Clustering Algorithms

1. k-Means Clustering

- **Definition:**
  - k-Means is a popular partitioning method that divides data into “k” clusters, where each data point belongs to the cluster with the nearest mean.

Common Clustering Algorithms

1. k-Means Clustering

Algorithm Steps:

1. Choose the number of clusters kkk.
2. Initialize kkk centroids randomly.
3. Assign each data point to the nearest centroid.
4. Recalculate the centroids based on the new cluster assignments.
5. Repeat steps 3 and 4 until centroids no longer change.

Common Clustering Algorithms

1. k-Means Clustering

Choosing the Optimal Number of Clusters (Elbow Method):

- The Elbow Method involves plotting the sum of squared distances (inertia) and finding the "elbow" point, where adding another cluster no longer significantly reduces the inertia.

Common Clustering Algorithms

1. k-Means Clustering

Pros and Cons of k-Means:

- **Pros:** Simple and efficient, works well with spherical clusters.
- **Cons:** Sensitive to initial centroids, requires specifying kkk, doesn’t work well with non-spherical clusters.

Common Clustering Algorithms

1. k-Means Clustering

Implementation in Python

```
from sklearn.cluster import KMeans
kmeans = KMeans(n_clusters=3, random_state=0)
kmeans.fit(X)
labels = kmeans.predict(X)
```

2. Hierarchical Clustering

Definition: Hierarchical clustering creates a hierarchy of clusters using either an agglomerative (bottom-up) or divisive (top-down) approach.

2. Hierarchical Clustering

Types of Hierarchical Clustering:

- **Agglomerative Clustering:** Starts with each data point as a separate cluster and merges the closest pairs iteratively.
- **Divisive Clustering:** Starts with all data points in one cluster and splits them iteratively.

2. Hierarchical Clustering

Dendrograms:

A dendrogram is a tree-like diagram that represents the hierarchy of clusters. The height of each “split” in the dendrogram represents the distance or similarity between clusters.

Common Clustering Algorithms

2. Hierarchical Clustering

Linkage Criteria:

- **Single Linkage:** Merges clusters based on the minimum distance between data points.
- **Complete Linkage:** Merges clusters based on the maximum distance between data points.
- **Average Linkage:** Merges clusters based on the average distance between data points.

2. Hierarchical Clustering

Pros and Cons of Hierarchical Clustering:

- **Pros:** No need to specify the number of clusters, produces a tree-based view of clusters.
- **Cons:** Computationally intensive, less effective with large datasets.

Common Clustering Algorithms

2. Hierarchical Clustering

Implementation in Python

```
from sklearn.cluster import AgglomerativeClustering
hc = AgglomerativeClustering(n_clusters=3, affinity='euclidean', linkage='ward')
hc.fit(X)
labels = hc.labels_
```

Common Clustering Algorithms

3. Density-Based Spatial Clustering of Applications with Noise (DBSCAN)

Definition: DBSCAN is a density-based clustering algorithm that identifies clusters based on areas of high density and can handle noise by marking it as outliers.

Common Clustering Algorithms

3. Density-Based Spatial Clustering of Applications with Noise (DBSCAN)

Key Concepts:

- **Core Points:** Points with at least “min_samples” points within a radius “eps.”
- **Border Points:** Points within the neighborhood of a core point but with fewer than “min_samples” neighbors.
- **Outliers:** Points that are neither core nor border points.

Common Clustering Algorithms

3. Density-Based Spatial Clustering of Applications with Noise (DBSCAN)

Parameters:

- **eps:** The radius within which to search for neighbors.
- **min_samples:** The minimum number of points required to form a dense region.

Common Clustering Algorithms

3. Density-Based Spatial Clustering of Applications with Noise (DBSCAN)

Pros and Cons of DBSCAN:

- **Pros:** No need to specify the number of clusters, effective for clusters with arbitrary shapes, robust to noise.
- **Cons:** Struggles with varying densities, sensitive to “eps” and “min_samples.”

Common Clustering Algorithms

3. Density-Based Spatial Clustering of Applications with Noise (DBSCAN)

Implementation in Python

```
from sklearn.cluster import DBSCAN
dbscan = DBSCAN(eps=0.5, min_samples=5)
dbscan.fit(X)
labels = dbscan.labels_
```

Common Clustering Algorithms

4. Gaussian Mixture Models (GMM)

Definition: GMM assumes that data points are generated from a mixture of several Gaussian distributions. Each component (cluster) in the GMM is represented by a Gaussian distribution with a specific mean and covariance.

Common Clustering Algorithms

4. Gaussian Mixture Models (GMM)

Algorithm Steps:

- GMM uses the Expectation-Maximization (EM) algorithm to find the parameters (mean, covariance) that best fit the data to a mixture of Gaussians.

Common Clustering Algorithms

4. Gaussian Mixture Models (GMM)

Soft Clustering:

GMM assigns probabilities to each data point, indicating the likelihood of belonging to each cluster, allowing for soft clustering.

Common Clustering Algorithms

4. Gaussian Mixture Models (GMM)

Implementation in Python

```
from sklearn.mixture import GaussianMixture
gmm = GaussianMixture(n_components=3, random_state=0)
gmm.fit(X)
labels = gmm.predict(X)
```

Evaluating Clustering Performance

Why Evaluation is Challenging in Clustering:

Since clustering is unsupervised, there are no true labels to compare the clusters against, making evaluation more complex.

Evaluating Clustering Performance

Common Evaluation Metrics:

1. Inertia (Sum of Squared Distances):

Measures the sum of squared distances between each data point and its assigned cluster center (used in k-means).

Evaluating Clustering Performance

Common Evaluation Metrics:

2. Silhouette Score:

Measures how similar a data point is to its assigned cluster compared to other clusters.

$$\text{Silhouette Score} = \frac{b - a}{\max(a, b)}$$

a: Mean intra-cluster distance.

b: Mean nearest-cluster distance.

Interpretation: A score closer to +1 indicates good clustering, while a score near 0 indicates overlapping clusters.

Evaluating Clustering Performance

Common Evaluation Metrics:

3. Davies-Bouldin Index:

Measures the average similarity ratio of each cluster with its most similar cluster. A lower Davies-Bouldin Index indicates better clustering.

Evaluating Clustering Performance

Common Evaluation Metrics:

4. Dunn Index:

Measures the ratio of minimum inter-cluster distance to maximum intra-cluster distance. A higher Dunn Index indicates better-defined clusters.

Implementation Example of Silhouette Score

```
from sklearn.metrics import silhouette_score
score = silhouette_score(X, labels)
```

Exercise 1: Implementing k-Means, DBSCAN, and GMM

- ❑ Load a dataset (e.g., Scikit-learn's `make_blobs` or `Iris` dataset).
- ❑ Apply k-means, DBSCAN, and GMM clustering on the dataset.
- ❑ Visualize the clusters using Matplotlib.